

Definitional Equalities for Substitutions using Rewrite Rules

Anonymous Author(s)

Abstract

Choosing a representation of variable binding is a central problem in mechanizing programming language metatheory. De Bruijn indices provide a canonical representation, but developments built on them require numerous lemmas about how substitutions compose and how they pass under binders, and every one of those lemmas has to be applied by hand or by a tactic.

The σ -calculus makes parallel substitutions first-class and normalizes them by rewriting. Modeling substitutions as functions turns its rules into theorems, which libraries such as AUTOSUBST exploit, but the resulting equalities stay propositional. We register the σ -calculus as rewrite rules in Agda, so that substitutions are normalized during conversion by the type checker rather than by a tactic.

We first solve the issue that the rules are provable only if the operations reduce, and eligible for rewriting only if the same definitions are stuck. We then identify which σ -calculus variant meets both requirements, and derive a rewrite system where renamings stay first-class, because a proof assistant needs them to define substitution by structural recursion.

A code generator emits the whole development from a syntax description. Every rule is registered together with a proof of the equality it installs, so logical consistency is preserved, and confluence of the resulting system keeps the type checker sound. To test the rules at scale we solve the POPLmark and POPLmark Reloaded challenges. We see this as groundwork for intrinsically typed formalizations of languages with a more complex type structure.

Keywords: Explicit Substitutions, Term Rewrite Systems, Agda

1 Introduction

Handling variable binding remains a central challenge in mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and boilerplate. Many representations have been proposed to contain them, among them locally nameless binders [12], nominal syntax [39, 40], higher-order abstract syntax [30], and de Bruijn indices [17]. De Bruijn syntax makes α -equivalent terms syntactically identical, and the syntax itself is easy to define. A development built on de Bruijn indices still requires a long

list of lemmas about substitution composition and about the interaction of substitution with binders.

On paper, substitution usually replaces a single variable. In a proof assistant one instead works with parallel substitutions, which map variables to terms and replace all of them simultaneously. The σ -calculus [1] makes such maps first-class objects with primitive syntactic operations for identity, weakening, extension and composition, together with a rewrite system that normalizes expressions built from them. The de Bruijn algebra [34] is the canonical semantic model for those operations, a two-sorted algebra over de Bruijn terms and substitutions, with substitutions modeled as total functions from variables to terms. In that model the rules are provable theorems, which is what makes them usable for equational reasoning in proof assistants. AUTOSUBST [35] and its successor AUTOSUBST 2 [38] generate the primitive operations, the proofs of the laws, and a tactic that normalizes substitutions inside a goal from a syntax description.

The equalities obtained by AUTOSUBST are propositional. Normalization is left to the user, so every goal that mentions a substitution has to be brought into normal form before it can be closed. In Rocq this is close to automatic, because AUTOSUBST supplies a tactic that performs the whole normalization procedure, but what the tactic produces is still a propositional equality. Recent support for user-defined rewrite rules in Agda [14, 15] and Rocq [26] hands normalization to the type checker. A proof of a propositional equality can be registered as a rule, and the type checker then reduces its left-hand side to its right-hand side, so the equality holds definitionally. The difference can be large for intrinsically typed syntax, when substitutions appears in a type index and every use of a propositional law becomes a transport [10].

Consistency of rewrite rules is retained as long as the registered rules are supported by proofs and the resulting system is confluent [15]. Agda applies a rewrite rule by matching a stuck term (i.e. an irreducible term that is headed by a defined function or a constructor) against the left-hand side of the registered proof [3].

Whether the left-hand side of a rule from the σ -calculus can be matched by type checker is decided by how the operations in it are defined. The same choice decides whether that law is provable. Modeled as functions from variables to terms or as vectors of terms, the σ -calculus operations reduce, and every law is provable. Under that same reduction, the left-hand sides for some rules are not stuck, and these laws cannot be registered for rewriting.

CPP '27,

2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

If we decide to model the operations as constructors, every left-hand side is stuck by construction, but we cannot derive proofs for the laws. Wadler [41] takes a minimal version of the constructor approach, making only weakening a constructor in the syntax, thereby successfully avoiding renamings. But weakening then does not push inwards into terms, and the definitional equalities that would push it there are unrecoverable.

We choose a definition with an underlying model whose laws are provable, and block their reduction artificially. Agda's opaque definitions [20] give the required control over the reduction behavior of definitions. A function declared opaque does not reduce outside its block, and an unfolding instruction lets a later block see through it again. Hence we can see the operations as ordinary functions where the proofs need them and as stuck definitions where the rules need to match them.

Opacity settles how the operations are defined, and leaves open which σ -calculus variant can be safely registered as rewrite rules. Registering safely means that every rule comes with a proof to preserve logical consistency, and that the rule set as a whole is confluent to keep the type checker sound. To make the matter more complicated, we also want renamings to be first class, because a proof assistant needs them to define substitution by structural recursion.

Contributions

- **A σ -calculus.** We give an σ -calculus variant with first-class renamings, that we can make definitional in a multi-sorted, intrinsically scoped setting using Agda. Every rule is proved before it is registered, hence logical consistency is preserved.
- **A metatheoretic study of the rewrite system.** Agda's built-in checker can validate local confluence, and using an external checker we can prove termination and confluence of the first-order export, which keeps the type checker sound.
- **A code generator.** We supply a script that emits the entire infrastructure from a syntax description.
- **An evaluation of the approach.** We solve the POPLmark Challenge [9], Parts 1A, 1B, 2A and 2B, and POPLmark Reloaded [2], on definitions and the rewrite system generated by the script. Only one substitution fact remains propositional: a non-trivial substitution that is a renaming in disguise.

Structure

In section 2 we explain the intrinsically scoped syntax setting we work in and define renaming and substitution over it. In section 3 we state the conditions Agda puts on a rewrite rule, identify the σ -calculus variant that meets them, derive the rules that follow with first class renamings, and establish confluence and termination. In section 4 we exercise the system on the metatheory of System F, explain how to

generate the infrastructure from a syntax description, and run it at scale on the POPLmark challenge. Section 5 collects future work, section 6 discusses related work, and section 7 concludes.

2 Syntax and Substitution

This section only establishes the required definitions. The development is carried out in Agda throughout. We give the multi-sorted and intrinsically scoped syntax of the paper (section 2.1) and define renaming and substitution over it (section 2.2). Two features of these definitions are important for what section 3 can register as rewrite rules. Variables form a sort of their own, generated by zero and successor, as is standard. Renamings are defined separately, because substitution cannot be defined structurally recursively without them.

2.1 Multi-Sorted and Scoped Syntax

Many programming languages feature several syntactic categories. For instance, System F [19, 31] has three: expressions, types, and kinds. Each usually gets its own datatype definition. Figure 1 (left) shows the standard intrinsically scoped implementation, where types are indexed by the number of type variables and expressions by the numbers of type and expression variables. Variables are correct-by-construction de Bruijn indices of type `Fin n`, the natural numbers smaller than n . Thus, scoped syntax rules out scoping errors without committing to full intrinsic typing.

The standard representation forces one substitution operation per pair of sorts, such as type-in-type, type-in-expression and expression-in-expression. This is the representation AUTOSUBST 2 chooses, and it explains why their rewrite system needs to be extended with vector substitutions, which bundle the substitutions over the different sorts and come with normalization rules of their own.

To avoid this combinatorial explosion we unify the syntax along two orthogonal dimensions, shown in fig. 1 (right). Variables and terms form a single datatype $S \vdash [m] s$, indexed by a `Mode` m , a target `Sort` s and a `Scope` S , a list of sorts. Variables of type $S \ni s$ are represented across all sorts at once, the length of S being the number of variables in scope and the position of s in S the de Bruijn index. Renamings and substitutions defined over this syntax then operate uniformly across sorts, and vector substitutions become unnecessary.

Recent work extends it to the maps over that syntax [7, 28], deriving renaming and substitution from a single traversal that abstracts over what a variable is mapped to, a variable or a term. Our syntax admits the same treatment, and some substitution laws would then follow uniformly for renamings and substitutions rather than twice in similar shape. We keep renaming and substitution apart for clarity of presentation, and because several of our rules treat the two differently.

```

221 data Kind : Set where
222   ★ : Kind
223
224 data Type (n : Nat) : Set where
225   ' _ : Fin n → Type n
226   ∀[α:_]_ : Kind → Type (suc n) → Type n
227   _⇒_ : Type n → Type n → Type n
228
229 data Expr (n m : Nat) : Set where
230   ' _ : Fin m → Expr n m
231   λx_ : Expr n (suc m) → Expr n m
232   Λα_ : Expr (suc n) m → Expr n m
233   ' _ : Expr n m → Expr n m → Expr n m
234   ' _ : Expr n m → Type n → Expr n m
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275

```

```

data Sort : Set where
  expr type kind : Sort
Scope = List Sort
data Mode : Set where V T : Mode
data _⊢[_]_ : Scope → Mode → Sort → Set
_⊢_ = _⊢[ T ]_
_⊢_ = _⊢[ V ]_
data _⊢[_]_ where
  zero : (s :: S) ⊢ s
  suc : S ⊢ s → (s' :: S) ⊢ s
  ' _ : S ⊢ s → S ⊢ s
  λx_ : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_ : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  ' _ : S ⊢ expr → S ⊢ expr → S ⊢ expr
  ' _ : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_ : S ⊢ type → S ⊢ type → S ⊢ type
  * : S ⊢ kind

```

Figure 1. System F as classically scoped syntax (left) and as the multi-sorted scoped syntax we use (right)

2.2 Renaming and Substitution

Instantiation, the operation that applies a substitution to a term, when defined by mutual recursion with composition is not structurally recursive. As is standard, we follow Adams [4] and define instantiation with renamings first. Renamings then run through the whole development, and proofs about them need the same reasoning we want to automate for substitutions. We will later follow AUTOSUBST 2 and put both into the equational theory and relate the two, when possible.

We model renamings and substitutions as functions in fig. 2. A renaming ($_ \rightarrow^R _$) takes every variable to a variable of the same sort, changing only the scope. A substitution ($_ \rightarrow^S _$) takes it to a term of that sort instead. We call either of them a *map* where the distinction does not matter. A map then has a mode as well, the mode of its images, **V** for a renaming and **T** for a substitution. The operations over maps are functions as well, and all of them are placed in an **opaque** block. They stay stuck, and we unfold them only in the proofs that need their computational behavior.

Modeling maps as functions is a choice. A vector of variables or terms, one per variable in the domain scope, would do as well, and would need the same opacity. Extension then is a constructor and stuck by itself, but every other operation is still a definition that unfolds. We choose the function model because we find it easier to think about, and accept that in return we then need to postulate function extensionality to work with equalities between maps. We return to the vector model in section 4.3.

The identity (id^R) takes every variable to itself. Weakening (wk^R) takes a variable to its successor, and takes the sort that is weakened by explicitly, because Agda frequently fails to infer it. Extension ($_ \rightarrow^R _$) adds an image for a newly bound variable, and lifting ($_ \uparrow^R _$) fixes that variable and weakens the images of the rest, which is what lets a renaming pass under a binder.

Instantiation ($_ \llbracket _ \rrbracket^R$) traverses a term and lifts whenever it enters a binder. It is defined on variables as well, so that a lookup is written $x \llbracket \xi \rrbracket^R$ rather than as the application $\xi _ x$. We only show three of its clauses, for a variable, a lambda binding and an application, and omits the five that are evident congruences. Composition ($_ \circ^R _$) looks a variable up in the first renaming and renames the result with the second, hence it's only ordinary function composition in disguise.

The coercion operator ($_ \langle _ \rangle$) embeds a renaming in a substitution, wrapping each image variable in the variable constructor. The remaining substitution operations are the same one level up, with terms in place of variables, and the figure lists them in that order. Extension ($_ \rightarrow^S _$) reads as before. Lifting ($_ \uparrow^S _$) weakens the images of the rest with the weakening renaming, which is what keeps the recursion structural. Instantiation ($_ \llbracket _ \rrbracket^S$) and composition ($_ \circ^S _$) read as before as well, except that a lookup now returns a term. The identity (id^S) and weakening (wk^S) substitutions are non-opaquely defined in terms of the coercion operator.

$\rightarrow^R : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^R S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \ni s$

opaque

$\text{id}^R : S \rightarrow^R S$

$\text{id}^R _ x = x$

$\text{wk}^R : \forall s' \rightarrow S \rightarrow^R (s' :: S)$

$\text{wk}^R _ x = \text{succ } x$

$_ \cdot^R : S_2 \ni s \rightarrow S_1 \rightarrow^R S_2 \rightarrow (s :: S_1) \rightarrow^R S_2$

$(x \cdot^R \xi) _ \text{zero} = x$

$(_ \cdot^R \xi) _ (\text{succ } x) = \xi _ x$

$_ \uparrow^R : S_1 \rightarrow^R S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^R (s :: S_2)$

$(\xi \uparrow^R _) _ \text{zero} = \text{zero}$

$(\xi \uparrow^R _) _ (\text{succ } x) = \text{succ } (\xi _ x)$

$_ \llbracket^R : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^R S_2 \rightarrow S_2 \vdash [m] s$

$_ \llbracket^R \{m = V\} x \xi = \xi _ x$

$(_ x) \llbracket \xi \rrbracket^R = _ (x \llbracket \xi \rrbracket^R)$

$(\lambda x e) \llbracket \xi \rrbracket^R = \lambda x (e \llbracket (\xi \uparrow^R _) \rrbracket^R)$

$(e_1 \cdot e_2) \llbracket \xi \rrbracket^R = (e_1 \llbracket \xi \rrbracket^R) \cdot (e_2 \llbracket \xi \rrbracket^R)$

-- ...

$_ \circ^R : S_1 \rightarrow^R S_2 \rightarrow S_2 \rightarrow^R S_3 \rightarrow S_1 \rightarrow^R S_3$

$(\xi_1 \circ^R \xi_2) _ x = (\xi_1 _ x) \llbracket \xi_2 \rrbracket^R$

$\rightarrow^S : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set}$
 $S_1 \rightarrow^S S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \vdash s$

opaque

$\langle _ \rangle : S_1 \rightarrow^R S_2 \rightarrow S_1 \rightarrow^S S_2$

$\langle \xi \rangle _ x = _ (x \llbracket \xi \rrbracket^R)$

$_ \cdot^S : S_2 \vdash s \rightarrow S_1 \rightarrow^S S_2 \rightarrow (s :: S_1) \rightarrow^S S_2$

$(t \cdot^S \sigma) _ \text{zero} = t$

$(t \cdot^S \sigma) _ (\text{succ } x) = \sigma _ x$

$_ \uparrow^S : S_1 \rightarrow^S S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^S (s :: S_2)$

$(\sigma \uparrow^S _) _ \text{zero} = _ \text{zero}$

$(\sigma \uparrow^S _) _ (\text{succ } x) = (\sigma _ x) \llbracket \text{wk}^R _ \rrbracket^R$

$_ \llbracket^S : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^S S_2 \rightarrow S_2 \vdash s$

$_ \llbracket^S \{m = V\} x \sigma = \sigma _ x$

$(_ x) \llbracket \sigma \rrbracket^S = \sigma _ x$

$(\lambda x e) \llbracket \sigma \rrbracket^S = \lambda x (e \llbracket (\sigma \uparrow^S _) \rrbracket^S)$

$(e_1 \cdot e_2) \llbracket \sigma \rrbracket^S = (e_1 \llbracket \sigma \rrbracket^S) \cdot (e_2 \llbracket \sigma \rrbracket^S)$

-- ...

$_ \circ^S : S_1 \rightarrow^S S_2 \rightarrow S_2 \rightarrow^S S_3 \rightarrow S_1 \rightarrow^S S_3$

$(\sigma_1 \circ^S \sigma_2) _ x = (\sigma_1 _ x) \llbracket \sigma_2 \rrbracket^S$

$\text{id}^S : S \rightarrow^S S$

$\text{id}^S = \langle \text{id}^R \rangle$

$\text{wk}^S : \forall s' \rightarrow S \rightarrow^S (s' :: S)$

$\text{wk}^S s' = \langle \text{wk}^R s' \rangle$

Figure 2. The operations for renaming (left) and for substitution (right)

3 Rewrite Rules for Renamings and Substitutions

We now search for a σ -calculus variant that is eligible for rewriting. What we require is a finite set of rules that is confluent on metavariables, and not every variant provides one for the definitions we gave in the previous section. The variants are stated for substitutions alone, so the rules for renamings have to be worked out as well. We state the conditions Agda puts on a rewrite rule in section 3.1, identify a variant that meets them in section 3.2, derive the rewrite system that follows with first class renamings in section 3.3, and establish confluence and termination in section 3.5. Throughout, the multi-sorted scoped System F is the running example, but the construction applies to any multi-sorted scoped syntax.

3.1 Rewrite Rules in Agda

User-defined rewrite rules [14, 15] extend Agda's normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules. They can be seen as a middle ground between intensional and extensional

type theory, where type checking stays decidable as long as the rules are confluent and terminating [15].

An equality proof of the shape $f p_1 \dots p_n \equiv v$ is eligible for rewriting under three conditions.

- f is a postulate, a defined function or a constructor.
- The left-hand side is stuck on f , that is, the application does not reduce.
- Every variable bound by the rule occurs on the left-hand side, and occurs there in pattern position.

The last condition serves matching. A bound variable appears in pattern position if it is either not applied at all, or applied to distinct bound variables and to nothing else. Applying a rule means matching its left-hand side against a stuck term, and that match has to determine every variable the rule binds. Matching is higher-order [3], that is, a variable bound by the rule may stand for a function and occur applied on the left-hand side. In general that problem is undecidable, and the restriction to pattern position makes it decidable.

For example, renaming a term by the identity leaves it unchanged. The law is proved by induction on the term, so it does not hold definitionally on its own:

```

right-idR : ∀ (t : S ⊢ s) → t [ idR ]R ≡ t
right-idR (‘ x) = refl
right-idR (t1 · t2) = cong2 _ _ (right-idR t1) (right-idR t2)
-- ...

```

It meets the three conditions: the left-hand side is stuck on the defined function $_[]^R$, because no clause matches a term using a variable pattern, and its only bound variable t occurs there unapplied. Once it is registered via $\{-\# \text{REWRITE right-id}^R \#\}$, Agda treats $t [\text{id}^R]^R$ as definitionally equal to t :

```

_ : ∀ {t : S ⊢ s} → t [ idR ]R ≡ t
_ = refl

```

Beyond the local restrictions, the global set of rewritten equations have to be well behaved. We want to preserve two guarantees of the proof assistant. The type checker is sound only if conversion preserves types, and this needs confluence of all registered rewrite rules with respect to the already existing reductions. Logical consistency is preserved when all rules are instantiated with propositional equality proofs [15, Sect. 3 and 6.4]

We prove all rules before registering them. Confluence is decided on the critical pairs of the rule set. The terms to which two reductions/rewrites apply, need to reduce to a common term. A rule has to be joinable against every rule other rule, and against the reduction Agda performs on its own, hence a rule interacts with the defining clauses of every function it mentions, which is why we keep the σ -calculus operations opaque.

3.2 Choosing the Correct Variant

The σ -calculus exists in several variants, and they differ on properties the registration as rewrite rules depends on. A $\lambda\sigma$ -calculus additionally includes the β and other reduction rules of the object language, which in our setting belong to the user's language description and lie outside the theory we register. Our rules deal only with substitutions. We forget about renamings for now, so for the purpose of this subsection id^S and wk^S are constants of the calculus.

The original σ -calculus. All variants share a set of rules that goes back to the original calculus of Abadi et al. [1], where lifting is defined in terms of the other operators: $\sigma \uparrow^S s = (\text{‘ zero}) \cdot^S (\sigma \circ^S \text{wk}^S s)$. We group them into four categories and name them after Stark et al. [38].

- Reduction laws push an instantiation through term constructors. There is one for every constructor of the syntax, as in

$$(\lambda x e) [\sigma]^S = \lambda x (e [\sigma \uparrow^S s]^S)$$

They are exactly the clauses of the instantiation operator of section 2.2.

- The defining clauses of the operations evaluate a substitution at a variable, so they fix the meaning of extension, weakening and lifting, for example

$$\text{zero} [t \cdot^S \sigma]^S = t$$

- Inference laws govern the interplay of composition, extension and weakening, which include associativity of composition and distributivity of composition over extension:

$$(\sigma_1 \circ^S \sigma_2) \circ^S \sigma_3 = \sigma_1 \circ^S (\sigma_2 \circ^S \sigma_3)$$

$$(t \cdot^S \sigma_1) \circ^S \sigma_2 = (t [\sigma_2]^S) \cdot^S (\sigma_1 \circ^S \sigma_2)$$

- The monad laws relate instantiation to composition. The identity substitute leaves a term untouched and compositionality fuses two instantiations into one:

$$t [\text{id}^S]^S = t$$

$$(t [\sigma_1]^S) [\sigma_2]^S = t [\sigma_1 \circ^S \sigma_2]^S$$

In our model, where substitutions are functions, the reduction laws and the defining clauses of the operations hold by computation. The monad laws are proved by induction on the term and the inference laws are proved using function extensionality.

The original σ -calculus establishes confluence on ground terms only. That does not suffice for a rewrite system inside a proof assistant, because a rule applies wherever the type checker finds a match and the terms it encounters contain metavariables.

The σ_{SP} -calculus. Curien et al. [16] repair this by adding new rules. Schäfer et al. [34] later show that the calculus is complete for the model in which substitutions are functions from variables to terms, so every equation that holds is derivable from the rules, and Stark [37] tabulates them. It is the version used in AUTOSUBST and thus the obvious candidate for this work. Unfortunately it clashes with our inductive definition of variables.

The new rules are the two η -laws. The general one equates a substitution with its value at the first variable extended by its restriction to the remaining ones, $(\text{zero} [\sigma]^S) \cdot^S (\text{wk}^S s \circ^S \sigma) = \sigma$, and its instance at the identity gives $(\text{‘ zero}) \cdot^S \text{wk}^S s = \text{id}^S$. They can be seen as surjective pairing for substitutions.

The left-hand side of η mentions σ twice, under a lookup and under a composition, so it is not left-linear. Every rule that reduces one of them and not the other opens a critical pair. Inductive variables force the law $x [\text{wk}^S s]^S = \text{‘ (suc } x)$, because with the variables generated by zero and suc , the weakened variable and $\text{suc } x$ are two terms for the same variable, and pattern matching on variables needs the constructor form to be the normal form.

Now instantiate σ in η with a weakening, which opens the pair in the top row of fig. 3. No rule collapses the extension on its right, because the only rules matching an extension

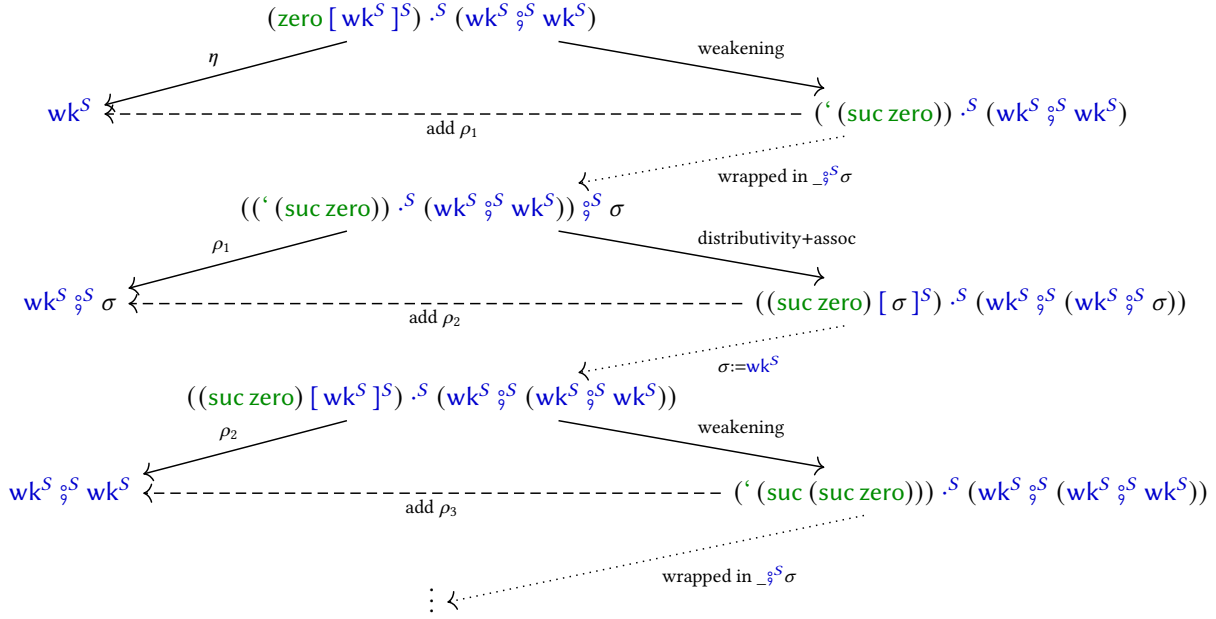


Figure 3. Attempt to complete the η -law against inductive variables

are the η -laws and both need the head to be **zero**. Joining it means adding the rule ρ_1 of the figure, and that rule has an extension on its left-hand side. Distributivity fires on an extension that is the left operand of a composition, so ρ_1 and distributivity overlap and the middle row is their pair. The rule ρ_2 that joins that one carries a lookup in the head, so instantiating σ with a weakening evaluates the lookup and reaches the bottom row, which is the top row with one more **suc**.

Observation 1. Inductive variables and the η -laws cannot coexist. Together they leave non-joinable critical pairs that no finite completion closes.

σ_{SP} has no **suc**. The n -th variable is represented as **zero** under n weakenings, so a lookup at a weakening is already a normal form, and with the operations declared as **postulates** its equations together with the η -laws pass the confluence checker. If we wish to eliminate the postulates to guarantee consistency and usability inside proofs, our only choice is weakening inside the term syntax, which is the work of Wadler on explicit weakenings. But then weakening does not push into terms, and we could not instantiate all the σ -calculus laws we require with proofs. The reduction law for application, $(t_1 \cdot t_2) [\sigma]^S = (t_1 [\sigma]^S) \cdot (t_2 [\sigma]^S)$, is one of them.

AUTOSUBST can combine inductive variables with the η -laws because a tactic has more freedom than a rewrite rule. It chooses where and in which order to apply an equation.

Confluence on metavariables is not a critical property for tactics.

Losing the η -laws is severe when lifting is defined in terms of the other operators, as it is in σ_{SP} . Lifting the identity is then the extension $(\text{zero}) .^S \text{wk}^S s$, and the η -law at the identity is the only rule that collapses it back to id^S . A lifted identity appears whenever a substitution passes under a binder, so we would be stuck frequently. Without the η -laws σ_{SP} therefore has nothing left to offer us, and we instead look at a variant that makes lifting a primitive operation.

The σ_{η} -calculus. This version comes from the same paper by Curien et al. [16] and is confluent on open terms. Its variables are generated inductively, and a weakened variable reduces to its successor, which is what we require. The drawback is that the η -laws hold only propositionally, so the calculus is not complete. We forfeit completeness in any case because we need renamings as a second kind of map, and the transfer between renamings and substitutions is incomplete.

A later paper by Hardin et al. [21] tabulates the 23 rules of σ_{η} , which fall into the four categories above together with a family of rules under a composition. Lifting is an operator of the calculus and requires laws of its own, for example $\text{id}^S \uparrow^S s = \text{id}^S$ and $(\sigma_1 \uparrow^S s) ;^S (\sigma_2 \uparrow^S s) = (\sigma_1 ;^S \sigma_2) \uparrow^S s$. The first is what the η -law had to supply where lifting is defined, and the second collapses nested binders.

None of these variants provides renamings, which are required for termination in proof assistants. The standard repair is to define renamings first [4]. AUTOSUBST 2 takes the further step of making renamings first-class and extending

the equational theory with rules that relate renamings and substitutions [38]. Their extended system also is no longer complete, and its confluence was not formally verified [37].

3.3 Extension with Renamings

The extended rule set includes a copy of $\sigma_{\uparrow}$ for the maps of each mode, with the reduction laws instantiated to the constructors of the signature. Additional rules connect the two modes. This section only presents the connection and the rules where renamings and substitutions need to be treated differently.

Renamings and substitutions are connected using the coercion operator $\langle _ \rangle$. Lifting a substitution weakens the images of the rest with the weakening renaming, which is what keeps the recursion structural, and id^S and wk^S are the coerced renamings $\langle \text{id}^R \rangle$ and $\langle \text{wk}^R s \rangle$. Due to that we need to make sure substitutions that are coerced renamings reduce to actual renamings for the appropriate rules to apply. For instance, the coincidence law [37] is oriented that way,

$$t[\langle \xi \rangle]^S = t[\xi]^R$$

A term instantiated with an embedded renaming is renamed directly. The remaining coercion rules keep embedded renamings closed under the substitution operations by moving $\langle _ \rangle$ outward, as in $\langle \xi_1 \rangle^S \langle \xi_2 \rangle^S = \langle \xi_1 \circ^R \xi_2 \rangle^S$, which we call $\langle _ \rangle$ -comp, and $\langle \langle \xi \rangle \uparrow^S s \rangle = \langle \xi \uparrow^R s \rangle$.

Because variables and terms are separate sorts, compositionality splits into eight laws where the σ -calculus has one, the four combinations of the two modes, each at terms and at variables. Each is registered in one of two directions. We call the direction that merges two instantiations into one instantiation under a composed map a fold, and the direction that takes a composed map apart into two applications a push. The eight compositionality rules point as follows:

$$\begin{array}{ll} (t[\xi_1]^R)[\xi_2]^R = t[\xi_1 \circ^R \xi_2]^R & \text{fold} \\ x[\xi_1 \circ^R \xi_2]^R = (x[\xi_1]^R)[\xi_2]^R & \text{push} \\ (t[\xi]^R)[\sigma]^S = t[\langle \xi \rangle \circ^S \sigma]^S & \text{fold} \\ x[\langle \xi \rangle \circ^S \sigma]^S = (x[\xi]^R)[\sigma]^S & \text{push} \\ (t[\sigma]^S)[\xi]^R = t[\sigma \circ^S \langle \xi \rangle]^S & \text{fold} \\ (x[\sigma]^S)[\xi]^R = x[\sigma \circ^S \langle \xi \rangle]^S & \text{fold} \\ (t[\sigma_1]^S)[\sigma_2]^S = t[\sigma_1 \circ^S \sigma_2]^S & \text{fold} \\ (x[\sigma_1]^S)[\sigma_2]^S = x[\sigma_1 \circ^S \sigma_2]^S & \text{fold} \end{array}$$

The direction follows what the first application produces. A renaming applied to a variable yields a variable, and there we push. Where the first map is a substitution the two sorts agree and the development registers them as a single mode-generic rule.

At variables the push is forced. Because weakening needs to be related to the successor, $(x[\xi]^R)[\text{wk}^R s]^R$ reduces to $\text{suc}(x[\xi]^R)$. Under a fold at variables it also reduces to

$x[\xi \circ^R \text{wk}^R s]^R$, and the only sound join is a rule matching $\xi \circ^R \text{wk}^R s$, a composition whose interesting part is the right operand, which conflict the right associativity. Figure 4 follows the attempt to complete the fold, where associativity moves the weakening one composition deeper at every step. Note that left associativity would have a similar issue.

Observation 2. Compositionality laws for variables with renamings on the left are registered as push, instead of a fold.

The push is mirroring the definition of composition. This is not a coincidence, because we want to apply the definition for composition, exactly when we can later apply a defining law for renamings to reduce it to an inductive variable.

3.4 The Full Rewrite System

The rest of the rules deal with the completion of rewrite system that resulted from our choices in the last two sections.

Table 1 counts the resulting system by group. Only the reduction laws depend on the signature. We use $|c|$ to denote the amount of constructors the syntax has. Every other group is the same for any syntax. The 4 groups above the middle line are the four categories of section 3.2, instantiated for the maps of each mode. The push and coercion rules below the line have no counterpart in $\sigma_{\uparrow}$.

Only the push rules remain to explain. The reason for their existence is that associativity does interfere with the push strategy. Associativity nests compositions to the right, so a rule whose left-hand side is a composition can stop matching. The interference law called interact, $\text{wk}^R s \circ^R (x \cdot^R \xi) = \xi$, shows the effect. Composing it with ξ' and re-associating gives $\text{wk}^R s \circ^R ((x \cdot^R \xi) \circ^R \xi')$, where interact requires an extension on the outside.

$\sigma_{\uparrow}$ restores the match with distributivity. We cannot have distributivity as an interference law, because it conflicts with the push strategy. The term $x[(y \cdot^R \xi_1) \circ^R \xi_2]^R$ reduces by push to $(x[y \cdot^R \xi_1]^R)[\xi_2]^R$ and by distributivity to $x[(y[\xi_2]^R] \cdot^R (\xi_1 \circ^R \xi_2))^R$. Registering the join of the

Table 1. The registered rules by group

#	group	content
2 * c	reduction laws	push an instantiation through a constructor, one per constructor and mode
10	defining clauses	evaluate a map at a variable, fixing extension, weakening and lifting
17	interference laws	the interplay of composition, extension, weakening and lifting, in both modes
6	monad laws	identity and compositionality, in all four mode combinations
14	push rules	completion rules for the push strategy
10	coercion	rules that collapse coercions into renamings

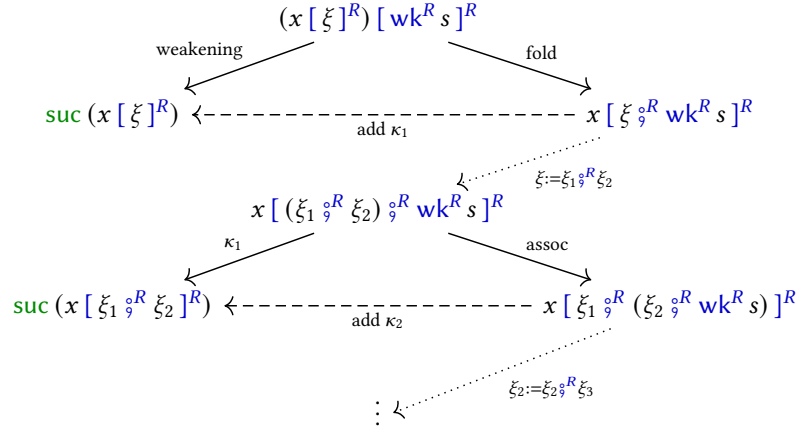


Figure 4. Attempt to complete the fold for renamings against weakening

two as a rule does not help. Its outer renaming is an unconstrained variable, so it matches a weakening and collides with $x [\text{wk}^R s]^R = \text{succ } x$, similar to the unjoinable pair that ruled out the fold at variables. The law for lifting, $(\xi \uparrow^R s) \circ^R (x \cdot^R \xi') = x \cdot^R (\xi \circ^R \xi')$, conflicts with push in the same way.

We give each affected rule a companion for the re-associated form instead. For interact,

$$\text{wk}^R s \circ^R ((x \cdot^R \xi) \circ^R \xi') = \xi \circ^R \xi'.$$

One level of nesting suffices, because associativity returns any deeper nesting to this shape. A rule is affected when its right operand is an extension or a lift, which re-association buries under a composition. Three renaming rules are affected, interact and the two laws that relate lifting to weakening and to composition.

The full rule set is available in the supplement. Because we have modeled maps as functions, all rules can be proven propositionally (assuming function extensionality) and then registered as rewrite rule.

3.5 Correctness

What remains is confluence. To this end, we rely on Agda's opt-in `--local-confluence-check`, which computes the critical pairs of the rewrite system. It checks that all critical pairs can be joined using other rewrite rules and internal reductions.

Agda also offers a global `--confluence-check`, which our system does not pass. Whenever two left-hand sides overlap, their most general unifier has to be a registered left-hand side as well. When the rule set includes associativity this will always fail, because associativity overlaps itself at a fourfold composition, and registering that instance only pushes the overlap to a longer one. No finite rule set with an associative composition meets the condition.

Agda does not check termination for rewrite rules. Under the global check termination is *not* a critical property [15]. Local confluence is the weaker property, and to conclude confluence using Newman's lemma [29], we need termination of the rewrite system. Proving termination has to leave Agda and we export the rewrite system as a first-order term rewriting system, with scope indices erased, and hand the resulting rules to an external prover. AProVE [18] successfully certifies termination and confluence. The exported rule set and the termination certificate can be found in the supplement.

Both Agda's local confluence check and the external termination check certify the rule set as it is seen from outside the `opaque` blocks, where the operations do not reduce. We therefore carry an invariant that the type checker cannot enforce.

Once a rewrite rule is registered, the opaque operations it mentions must never be unfolded again.

We unfold the definitions only to prove the rules, and we prove all of them before we register any of them as rewrites.

Together the two properties, the instantiation of the rules with propositional equalities and confluence, preserve logical consistency and subject reduction. Consistency makes the proofs meaningful and subject reduction keeps the type checker sound.

4 Implementation and Evaluation

We first exercise the rule set on the metatheory of System F (section 4.1). The rest of this section focuses on generalizing the method. We generate the infrastructure from a syntax description (section 4.2) and measure on the POPLmark Challenge how much substitution reasoning the rule set leaves (section 4.4).

4.1 Metatheory for System F

We show the rule set at work on a subject reduction proof for the System F version of fig. 1. Subject reduction rests on the substitution-preserves-typing lemma, that applying a well typed substitution to a well typed term preserves typing. That lemma rests in turn on the same statement for renamings. We look at two occurrences where normally substitution reasoning is required, one from a typing rule and one from a reduction rule. The full development can also be found in the supplement.

The first comes from the typing rule for type application. Instantiating a polymorphic type substitutes for the variable it binds, so the rule concludes at the single substitution $t' [t]_0$, our notation for $t' [t \cdot^S \text{id}^S]^S$.

$\vdash \bullet :$
 $\Gamma \vdash e : (\forall [\alpha : k] t') \rightarrow$
 $\Gamma \vdash t : k \rightarrow$
 $(k ::_t \Gamma) \vdash t' : k' \rightarrow$
 $\Gamma \vdash (e \bullet t) : (t' [t]_0)$

In the substitution-preserves-typing lemma this conclusion has to survive an outer σ . The goal is $(t' [t]_0) [\sigma]^S$, because that is what the rule concludes with σ applied to it. The sub-derivations instead supply $(t' [\sigma \uparrow^S s]^S) [t [\sigma]^S]_0$, because the body was typed under a lifted σ and the argument under σ alone. The two substitutions have to agree, which is the lemma we call **subst-commute**.

The second comes from call-by-value β -reduction.

$\beta\text{-}\lambda :$
 $\text{Val } e_2 \rightarrow$
 $((\lambda x e_1) \cdot e_2) \hookrightarrow (e_1 [e_2]_0)$

The typing rule for abstraction stores the result type of a function under **weaken**, the instantiation with the weakening renaming, **weaken** $t = t [\text{wk}^R s]^R$, because the body is typed in a longer context.

$\vdash \lambda :$
 $(t_1 ::_t \Gamma) \vdash e : (\text{weaken } t_2) \rightarrow$
 $\Gamma \vdash (\lambda x e) : (t_1 \Rightarrow t_2)$

That weakening is bookkeeping. Because scopes are shared over sorts, binding an expression variable also extends the scope that types are indexed by. Subject reduction for this redex therefore delivers the type $(\text{weaken } t_2) [e_2]_0$, while the goal is the plain t_2 the application concluded at. The weakening and the substitution have to cancel, which is **wk-cancel**.

Both are lemmas a de Bruijn development needs to state, prove and and apply by hand. Here both hold by **refl**.

wk-cancel : $\forall \{t : S \vdash s\} \{t' : S \vdash s'\} \rightarrow (\text{weaken } t) [t']_0 \equiv t$

wk-cancel = **refl**

subst-commute : $\forall \{t : (s' :: S_1) \vdash s\} \{t' : S_1 \vdash s'\}$

$\{\sigma : S_1 \rightarrow^S S_2\} \rightarrow$

$(t [(\sigma \uparrow^S s')^S] [t' [\sigma]^S]_0) \equiv (t [t']_0) [\sigma]^S$
subst-commute = **refl**

For **wk-cancel** the weakening is a renaming and the single substitution is an extension, so three rules suffice.

$(\text{weaken } t) [t']_0 = (t [\text{wk}^R s]^R) [t' \cdot^S \text{id}^S]^S$
 $\rightarrow t [\text{wk}^S s]^S (t' \cdot^S \text{id}^S)^S$ compositionality
 $\rightarrow t [\text{id}^S]^S$ interact
 $\rightarrow t$ identity

The middle step is the interference law that cancels a weakening against an extension. For **subst-commute** the two sides start apart and meet in the middle.

$(t [\sigma \uparrow^S s]^S) [t' [\sigma]^S]_0 = (t [\sigma \uparrow^S s]^S) [t' [\sigma]^S \cdot^S \text{id}^S]^S$
 $\rightarrow t [(\sigma \uparrow^S s) \circ^S (t' [\sigma]^S \cdot^S \text{id}^S)]^S$ compositionality
 $\rightarrow t [t' [\sigma]^S]^S (\sigma \circ^S \text{id}^S)^S$ lift-cons
 $\rightarrow t [t' [\sigma]^S]^S \sigma^S$ comp-id_r
 $(t [t']_0) [\sigma]^S = (t [t' \cdot^S \text{id}^S]^S) [\sigma]^S$
 $\rightarrow t [(t' \cdot^S \text{id}^S) \circ^S \sigma]^S$ compositionality
 $\rightarrow t [t' [\sigma]^S]^S (\text{id}^S \circ^S \sigma)^S$ distributivity
 $\rightarrow t [t' [\sigma]^S]^S \sigma^S$ comp-id_l

Three of the laws in them are interference laws we have not named so far. **lift-cons** composes a lifted map with an extension, where the bound variable takes the extended term and everything above it passes to the tail. **comp-id_l** and **comp-id_r** are the two unit laws of composition.

4.2 Code Generation

The code generation script from this section was produced with help from GenAI.

Nothing in section 3 is specific to System F. Of the seven groups in table 1, only the reduction laws depend on the signature, and they can be derived generically. We provide a script that turns a higher-order abstract syntax (HOAS) description [35] into Agda code with the rules ready for rewriting.

Figure 5 shows such a description, our running example extended with record types and pattern matching. It is the entire input needed to full rule set. A line beginning with % is passed through to the preamble of the generated module. Binders are function arguments, that specify the sort of the argument. Record labels are represented as strings. In general, external arguments can be written in quotes, and the generated instantiations carry them along without touching them. A pattern binds a variable number of expression variables at once. The match constructor takes the arity #n as its first argument and then binds that many variables, which is what the iterated binder $\text{expr} \wedge n$ means. AUTOSUBST 2 does the same for Rocq, emitting the scoped syntax together with the substitution lemmas that need induction over it.

```

991 % open import Data.String using (String)
992 kind : Type
993 type : Type
994 expr : Type
995 row : Type
996 pat : Type
997
998 -- the constructors for kind
999 star : kind
1000
1001 -- the constructors for type
1002 arr : type -> type -> type
1003 all : kind -> (type -> type) -> type
1004 rec : row -> type
1005
1006
1007 -- rows of labelled fields
1008 rnil : row
1009 rcons : "String" -> type -> row -> row
1010
1011 -- the constructors for expr
1012 app : expr -> expr -> expr
1013 lam : type -> (expr -> expr) -> expr
1014 tapp : expr -> type -> expr
1015 tlam : kind -> (type -> expr) -> expr
1016 proj : expr -> "String" -> expr
1017 match : #n -> expr -> pat -> (expr ^ n -> expr) -> expr
1018
1019 -- the constructors for pat
1020 pvar : pat
1021 ppair : pat -> pat -> pat

```

Figure 5. System F with record types and pattern matching

Several of AUTOSUBST 2's features come for free. Vector substitutions are unnecessary and mutually recursive definitions work unchanged, both because the syntax is multi-sorted. We do not support inline Agda code that depends on the syntax, such as lists or products of terms. We note that they can be encoded with additional sorts, at the cost of standard library support.

Variadic syntax is supported, i.e. a binder that binds a variable number of variables at once, such as `match` does. It needs a new lifting operation and a set of laws that supports it. The code generation includes a new lifting operator $\uparrow^*[_]$, that takes a sort and a count and iterates the ordinary lifting that many times. Additional laws support the new operator.

The script is a single Python file with strict type checking enabled. It includes a parser for HOAS descriptions and ad-hoc code generation, and it carries all boilerplate Agda code so that others can run it standalone. Beside the five languages of section 4.4 we generated a dozen files from the HOAS descriptions shipped with AUTOSUBST 2.

Although the model we use for maps is not interesting theoretically, it matters for performance. The script generates the whole rewrite infrastructure for maps modeled either as vectors or as functions.

4.3 Vectors vs. Functions

A vector-like data type for maps is the other candidate representation that we have not shown in this paper. Extension is a constructor, and identity, weakening, lifting and composition stay defined by recursion over the vector, so their clauses compete with the rules with respect to confluence exactly as the function model's do, and opacity is also needed to block their reduction. With vectors nothing is lost in terms of provability. Equality of vectors is equality of data, so that variant needs no function extensionality postulate.

The vector model also type checks faster. We measured the time on the $F_{<}$ signature with records and pattern matching, together with a large proof for transitivity of subtyping with narrowing and preservation and progress on top of it. The proof is 1,161 lines and is unchanged between the two models. Together definitions and proofs take about 23 s to type check with vectors against about 145 s with functions, a speedup of about six. Both figures were taken on a standard developer machine.

We can only guess why the speedup is so large. One possibility is that Agda optimizes matching against constructors, which here would only benefit the extension constructor. Another is that the higher-order nature of maps as functions leaks into the higher-order pattern unification that Agda performs to decide whether a rule matches a stuck term. That should not happen, because the operations we use are opaque, but maybe Agda still sees that the maps underneath are defined as functions.

The proof we measured on is part of the POPLmark challenge.

4.4 POPLmark and POPLmark Reloaded

The proofs from this section were produced with help from GenAI.

To test the construction at a scale where substitution reasoning hurts, we solve the POPLmark Challenge [9] and POPLmark Reloaded [2] including its sums extension. The development is roughly 8,400 lines of Agda, 6,100 of them excluding blanks and comments, of which 2,800 are generated infrastructure and 3,200 metatheory. Table 2 lists the languages it involves and what is proved for each.

The point of comparison is the substitution reasoning a de Bruijn development normally needs to do. For a fair comparison we use the Rocq solutions by the AUTOSUBST

Table 2. Coverage per language

Language	POPLmark	Generated LOC	Proof LOC
STLC, sums	Reloaded 1a--2b	504	728
F _{<}	1A, 2A	532	404
F _{<} , records, patterns	1B, 2B	715	818

authors, where the σ -calculus laws are applied by the `asimpl` tactic.

The η -laws, the ones we cannot register, are never needed. This is because lifting is a primitive operation rather than one defined in terms of the others.

Only one proof obligation for maps remains propositional. It comes up in Lemma 3.19 of POPLmark Reloaded, the extensionality lemma for strong normalization. The β -step there produces $b \llbracket x \rrbracket_0$, which has to meet an anti-renaming lemma stated for renamings, and that requires

$$t \llbracket x \rrbracket_0 = t \llbracket x \cdot^R \text{id}^R \rrbracket^R,$$

Its two sides are distinct normal forms in our rewrite system. The left one unfolds to $t \llbracket (x) \cdot^S \text{id}^S \rrbracket^S$, an extension, which coincidence cannot match. The rule that would bridge them is $(x) \cdot^S \langle \xi \rangle \rightarrow (x \cdot^R \xi)$, oriented $S \rightarrow R$ like the rest of the coercion group. Registering it leaves non-joinable pairs that close only if composition at a variable folds, against the push orientation of observation 2.

The same fact lies outside what `asimpl` normalizes in AUTOSUBST. There it is supplied by two separate tactics, `substify` and `renamify`, that provide strategies to move a map from one mode to the other.

5 Future Work

5.1 Opaque Definitions for Rewriting

Our construction with opaque blocks and rewrite rules suggests a general recipe for integrating normalization for suitable algebraic structures into a proof assistant: define operations for the structure; establish a confluent rewrite system over them; instantiate the structure with an underlying model and prove the laws inside `unfolding` blocks; and, with the operations never unfolded again, `REWRITE` the equations.

Classical sets are one candidate. Reasoning about them in Agda usually means lists up to permutation together with a uniqueness predicate, which is tedious and generates a great deal of uninteresting code. Their characteristic function supplies an adequate model, that is usually tedious to work with. Sets become $S = \mathcal{A} \rightarrow \text{Bool}$ for some $\mathcal{A} : \text{Set } \ell$, the operations become opaque functions

```
opaque
  () : S
  () a = false
```

$$\begin{aligned} _ \cup _ : S &\rightarrow S \rightarrow S \\ (A \cup B) a &= A a \vee B a \end{aligned}$$

and laws such as $(A \cup \emptyset) \equiv A$ become provable and registrable. The boolean ring interpretation [22] is a natural starting point for the rule set, but it needs rewriting modulo associativity and commutativity (AC). Dedukti [8] supports this and Agda does not, which is why think rewriting modulo AC would be a great addition to Agda.

5.2 Implications for Intrinsically Typed Syntax

Our construction already works for intrinsically typed syntax whose type index lives in `Set`, for example the simply typed λ -calculus. Richer types, where the type level has substitutions of its own, for example System F, require more work. Transport hell is the colloquial name for having to apply type-changing lemmas to terms that are already known to be equal. It sets in there, because substitutions that agree only propositionally force terms to be moved across equal types with the transfer lemma (`subst` in Agda). That cost is long-standing in intrinsic encodings [10].

Intrinsically typed System F has been formalized in Agda before [11], and a binary logical relation over it [33] shows what that costs. Type-in-type substitution and expression-in-expression substitution reasoning is not complicated there, but type-in-expression substitution runs into transport hell, and the authors spend thousands of lines with uninteresting substitution reasoning. We expect that, rewriting the σ -calculus rules on the syntax of the typing index removes the transports and it should generalize to other intrinsically typed formalizations.

6 Related Work

We have already discussed all the work on the σ -calculus we build on. We start with the one other development that formalizes a definitional substitution calculus without rewrite rules, then the theory of rewrite rules we depend on and the earlier attempts to register the laws with it. The generators for substitution machinery and the work on multi-sorted syntaxes follow.

Definitional substitution calculi. Kaposi and Pujet strictify the syntax [23], presenting the initial category with families so that substitutions are metatheoretical functions inside strict presheaves over the syntax of a dependent type theory. They work in observational type theory, although it seems that what the construction relies on is more definitional proof irrelevance than observational equality. Composition of substitutions is then composition of functions, so the laws hold by conversion and most of the transports that an indexed-inductive presentation forces disappear. Their strictified syntax has the induction principle of the initial model, which a shallow embedding of the same syntax does not. In their formalization, eleven of the twelve equations of the strictified category with families hold by reflexivity. The

Table 3. Comparison with strictified syntax

	this work	Kaposi and Pujet
goal	custom object-language metatheory	metatheory of type theory
axioms	none, in the vector model	funext, QIITs
syntax variables	intrinsically scoped inductive	intrinsically typed explicit weakening
laws hold by	proof, then registration as rewrites	definitional encoding
η -law	no	no
induction principle	native, computes definitionally	transported, computes propositionally
demonstrated on	POPLmark, five languages	canonicity for TT

twelfth law, the one they and we cannot make definitional, is the η -law.

Their goal is type theory in type theory, an intrinsically typed syntax for dependent types, and canonicity for it. Rewrite rules appear in their formalization only to emulate the metatheory they work in, the observational equality and the postulated quotient inductive-inductive types. Our goal is a framework that others instantiate for custom object languages, and we take rewrite rules as a tool we may use wherever their use stays consistent. Section 5.2 already discussed which intrinsically typed syntaxes we think our method can simplify, and also discussed that TT-in-TT for now is out of reach.

Table 3 shows the two developments side by side. Their syntax has weakening as a constructor, as Wadler’s does [41], so they don’t have to deal with inductive variables.

Chen, Nordvall Forsberg and Tsai [13] give an intrinsic representation of type theory in Cubical Agda as quotient inductive-inductive-recursive types, and obtain a syntax that needs no transports. Their substitution laws are path constructors, so reasoning with them transports along paths, and they report that transport hell reappears once the metatheory becomes more sophisticated.

Kaposi and Xie replace the parallel substitution calculus by a single substitution calculus with eight equations [24], and postulate its syntax as a quotient inductive-inductive type whose computation rules are added with rewrite rules. They register the equations of a postulated initial model, where we register laws proved in a model we construct.

Rewrite rules in proof assistants. We have discussed user-defined rewrite rules [14, 15, 26] and opaque definitions [20] throughout this work. Leray and Winterhalter’s never work on local rewrite rules [27] also rewrite σ -calculus laws, with support for first-class renamings in Rocq. They too model renamings and substitutions as functions, but cannot rewrite all laws against that model, because with local rewrite rules the instantiation of the equations must hold definitionally

rather than propositionally to maintain consistency. Rocq also has global rewrite rules [26], and we expect our approach to translate. Rocq, however, at the moment cannot instantiate rewrite rules with proofs, so the consistency guarantee would be lost.

Two further developments register σ -calculus laws and stop short of a confluent set. Wadler’s experiment with explicit weakening also registers rewrites for some of the σ -calculus laws but does not pass the confluence checker. Section 1 discussed what downsides that representation entails. The ABSTRACT BINDING TREES library has experimental support for rewriting with σ -calculus laws that misses rules and does not pass the confluence check either.

Substitution automation. That library builds an object language from a generic syntax and is used, for instance, in Siek’s work [36]. It shares a goal with AUTOSUBST, but supports only extrinsic scoping and single-sorted syntax, and focuses on generic theorems about substitution, without external code generation. We have discussed AUTOSUBST and related research [35, 37, 38] throughout this work. Other generators for binder boilerplate predate and accompany it. Needle & Knot [25] generates the boilerplate for complex binding forms, but does not aim at an equational theory of substitution. Benton and colleagues encode terms indexed by a typing environment and a type [10], the intrinsically typed counterpart of the scope-indexed syntax we use. The POPLmark Challenge [9] and POPLmark Reloaded [2] are the shared benchmarks for these developments, and section 4.4 reports our coverage of both.

Multi-sorted and uniform syntax. Generic programming approaches [5, 6] build syntax from general building blocks and abstract over renamings and substitutions, and the approach carries over to a scoped, multi-sorted setting [32]. Applying our rule set to such a universe of syntaxes would remove the need for external code generation, but is also more annoying to use. Saffrich generates the syntax-dependent lemmas by reflection, which could also replace our generator. Altenkirch, Burke and Wadler treat renamings and substitutions uniformly [7], following McBride [28], and thereby merge all four compositionality lemmas into one. Using their approach also lemma such as substitution-preserves-typing only need to be written once. Adapting our rules to that setting would need a second rewrite system on the type level, normalizing expressions in the lattice of *Modes* generated by $V \sqsubseteq T$, and in our experiments the interaction of the two layers made confluence hard to preserve.

7 Conclusion

We have turned the normalization procedure of the σ -calculus into definitional equalities in Agda. The operations must compute, so that the laws are provable, and stay inert, so that the rules match. Opaque definitions give both.

The theory we register is σ_{fl} instantiated once per mode, with the rules that first-class renamings force between the two. Which laws survive is forced, because native inductive variables and surjective pairing cannot coexist in one rewrite system. Agda certifies local confluence and AProVE termination, a generator emits it from a syntax description, and the POPLmark challenges exercise it at scale.

References

- [1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '90). Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712
- [2] Andreas Abel, Guillaume Allais, Aliya Hameer, Brigitte Pientka, Alberto Momigliano, Steven Schäfer, and Kathrin Stark. 2019. POPLMark Reloaded: Mechanizing Proofs by Logical Relations. *Journal of Functional Programming* 29 (2019), e19. doi:10.1017/S0956796819000170
- [3] Andreas Abel and Brigitte Pientka. 2011. Higher-Order Dynamic Pattern Unification for Dependent Types and Records. In *Typed Lambda Calculi and Applications*, Luke Ong (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 10–26.
- [4] Robin Adams. 2004. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs* (Jouy-en-Josas, France) (TYPES'04). Springer-Verlag, Berlin, Heidelberg, 1–16. doi:10.1007/11617990_1
- [5] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. 2021. A Type and Scope Safe Universe of Syntaxes with Binding: Their Semantics and Proofs. arXiv:2001.11001 [cs.PL] <https://arxiv.org/abs/2001.11001>
- [6] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs* (Paris, France) (CPP 2017). Association for Computing Machinery, New York, NY, USA, 195–207. doi:10.1145/3018610.3018613
- [7] Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. 2025. Substitution without copy and paste. <https://nottingham-repository.worktribe.com/output/51884690/substitution-without-copy-and-paste>. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)* (2025-07-19). ACM, Birmingham, UK, 17 pages.
- [8] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. 2023. Dedukti: a Logical Framework based on the $\lambda\Pi$ -Calculus Modulo Theory. arXiv:2311.07185 [cs.LO] <https://arxiv.org/abs/2311.07185>
- [9] Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics (TPHOLs 2005) (Lecture Notes in Computer Science, Vol. 3603)*. Springer, Berlin, Heidelberg, 50–65. doi:10.1007/11541868_4
- [10] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1–19. doi:10.1007/s10817-011-9219-0
- [11] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10
- [12] Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- [13] Liang-Ting Chen, Fredrik Nordvall Forsberg, and Tzu-Chun Tsai. 2026. Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) (CPP '26). Association for Computing Machinery, New York, NY, USA, 201–215. doi:10.1145/3779031.3779090
- [14] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:27. doi:10.4230/LIPIcs.TYPES.2019.2
- [15] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341
- [16] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675
- [17] N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0
- [18] Jürgen Giesl, Cornelius Aschermann, Marc Brockschmidt, Fabian Emmes, Florian Frohn, Carsten Fuhs, Jera Hensel, Carsten Otto, Martin Plücker, Peter Schneider-Kamp, Thomas Ströder, Stephanie Swiderski, and René Thiemann. 2017. Analyzing Program Termination and Complexity Automatically with AProVE. *Journal of Automated Reasoning* 58, 1 (2017), 3–31. doi:10.1007/s10817-016-9388-y
- [19] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis. Éditeur inconnu. <https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf>
- [20] Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. 2025. Controlling unfolding in type theory. *Mathematical Structures in Computer Science* 35 (2025), e38, 1–37. doi:10.1017/S0960129525100327
- [21] Thérèse Hardin, Luc Maranget, and Bruno Pagano. 1998. Functional runtime systems within the lambda-sigma calculus. *Journal of Functional Programming* 8, 2 (1998), 131–176. doi:10.1017/S0956796898003001
- [22] Jieh Hsiang. 1985. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence* 25, 3 (1985), 255–300. doi:10.1016/0004-3702(85)90074-8
- [23] Ambrus Kaposi and Loïc Pujet. 2025. Type Theory in Type Theory using a Strictified Syntax. *Proc. ACM Program. Lang.* 9, ICFP, Article 266 (Aug. 2025), 31 pages. doi:10.1145/3747535
- [24] Ambrus Kaposi and Szumi Xie. 2025. Type Theory with Single Substitutions. *Electronic Proceedings in Theoretical Computer Science* 431 (2025), 42–64. doi:10.4204/EPTCS.431.4
- [25] Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. 2016. Needle & Knot: Binder Boilerplate Tied Up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*. Springer-Verlag, Berlin, Heidelberg, 419–445. doi:10.1007/978-3-662-49498-1_17
- [26] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 255–297. doi:10.1007/978-3-030-33636-3_10

- Germany, 26:1--26:18. doi:10.4230/LIPIcs.ITP.2024.26
- [27] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704
- [28] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). <http://strictlypositive.org/ren-sub.pdf> Unpublished manuscript.
- [29] M. H. A. Newman. 1942. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics* 43, 2 (1942), 223--243. doi:10.2307/1968867
- [30] Frank Pfenning and Conal Elliott. 1988. Higher-order abstract syntax. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI '88)*. Association for Computing Machinery, New York, NY, USA, 199--208. doi:10.1145/53990.54010
- [31] John C. Reynolds. 1974. Towards a theory of type structure. In *Programming Symposium*, B. Robinet (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 408--425. <https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf>
- [32] Hannes Saffrich. 2024. Abstractions for Multi-Sorted Substitutions. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1--32:19. doi:10.4230/LIPIcs.ITP.2024.32
- [33] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development (Milan, Italy) (TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2--15. doi:10.1145/3678000.3678201
- [34] Steven Schäfer, Gert Smolka, and Tobias Tebbi. 2015. Completeness and Decidability of de Bruijn Substitution Algebra in Coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs (Mumbai, India) (CPP '15)*. Association for Computing Machinery, New York, NY, USA, 67--73. doi:10.1145/2676724.2693163
- [35] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359--374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf
- [36] Jeremy G. Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming* 31 (2021), e30. doi:10.1017/S0956796821000241
- [37] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq*. Ph.D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).
- [38] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (Cascais, Portugal) (CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166--180. doi:10.1145/3293880.3294101
- [39] Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. 2013. Nominal 2. *Archive of Formal Proofs* 2013 (February 2013), 1 pages. <https://isa-afp.org/entries/Nominal2.html>, Formal proof development.
- [40] Christian Urban and Christine Tasson. 2005. Nominal techniques in Isabelle/HOL. In *Proceedings of the 20th International Conference on Automated Deduction (Tallinn, Estonia) (CADE' 20)*. Springer-Verlag, Berlin, Heidelberg, 38--53. doi:10.1007/11532231_4
- [41] Philip Wadler. 2024. Explicit Weakening. *Electronic Proceedings in Theoretical Computer Science* 413 (Nov. 2024), 15--26. doi:10.4204/eptcs.413.2